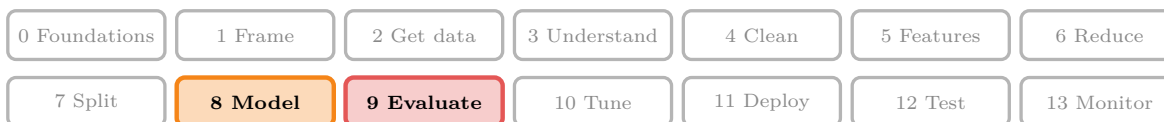


L1 and L2 Regularisation in Neural Networks

Contents

1. Overview	2
2. Prerequisites	3
3. Why neural networks overfit	3
4. Ways to reduce overfitting	5
5. The penalty term	5
5.1 L2 regularisation	6
5.2 L1 regularisation and both together	6
6. Why the weights shrink: weight decay	7
7. Regularisation in Keras	7
7.1 The data and the network	8
7.2 Without regularisation	8
7.3 With L2	9
7.4 The weights shrink	9
7.5 L1	11
7.6 Sparse or only small: L1 against L2	11
8. Summary	14
9. Sources	14
10. Key terms	14

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the [Course map](#).



- **Builds on:** Train-test split ([Note 13](#)); Vector magnitude, distance and scalar operations ([Note 361](#)); Multi-layer perceptron (MLP) ([Note 1009](#)); Gradient descent ([Note 1020](#)).
- **Leads to:** Hyperparameter tuning ([Note 1039](#)); Keras Tuner ([Note 1039](#)); Image classification with a CNN (cats vs dogs) ([Note 1049](#)); Data augmentation ([Note 1050](#)); Transfer learning (feature extraction and fine-tuning) ([Note 1053](#)); Keras functional API ([Note 1054](#)).
- **Compare with:** Polynomial regression ([Note 61](#)); Ridge regression ([Note 66](#)); Lasso regression ([Note 68](#)); Decision surface and boundary ([Note 91](#)); Underfitting ([Note 91](#)); Dropout ([Note 1025](#)).

1. Overview

Key point: Adding a penalty on the size of the weights to the loss pushes every weight towards 0 at each update. The network becomes simpler and overfits less. In Keras it is one argument per layer: `kernel_regularizer`.

Regularisation was taught for linear models: Ridge adds the squared coefficients to the loss, Lasso their absolute values (see the [ridge regression Note](#) and the [Lasso Note](#)). The same idea works for neural networks, where it is used whenever a network overfits.

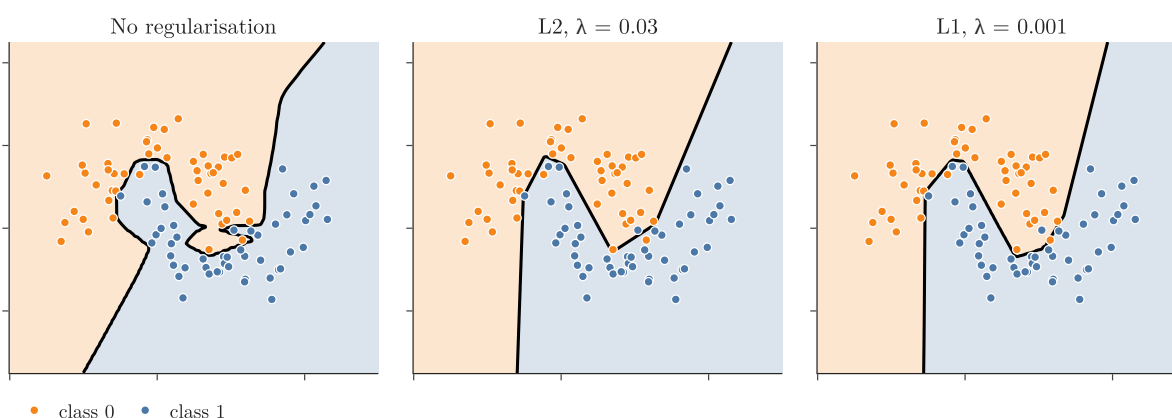


Figure 1: The same 128-128 network on 100 points of `make_moons`, trained for 2,000 epochs: without regularisation, with L2 and with L1.

Figure 1 shows the effect. This Note covers:

- why networks overfit (section 3);

- the ways to fight it (section 4);
- the penalty and why it shrinks the weights (sections 5 and 6);
- the Keras code with its results (section 7).

The Notebook (`notebook.ipynb`) runs every step.

2. Prerequisites

- The [ridge regression Note](#), the [ridge gradient descent Note](#) and the [Lasso Note](#).
- The [bias-variance Note](#): overfitting and underfitting.
- The [loss functions Note](#): loss versus cost function.

3. Why neural networks overfit

Key point: Each neuron can contribute one more line to the decision boundary. With hundreds of neurons, the boundary can bend around every training point.

Overfitting means a model does very well on the training data and poorly on new data: it memorises the training data, minor patterns included, instead of learning the concept (see the [bias-variance Note](#)). Like a student who learns every formula by heart without understanding: new questions in the exam go badly.

Complex models such as neural networks or very deep decision trees are prone to it, because their training aims to make no mistake on the training data. The main cause is the complexity of the network: many nodes, all connected to each other.

Figure 2 makes this concrete. A network with one hidden layer is trained on the same 100 points, with more and more neurons in that layer.

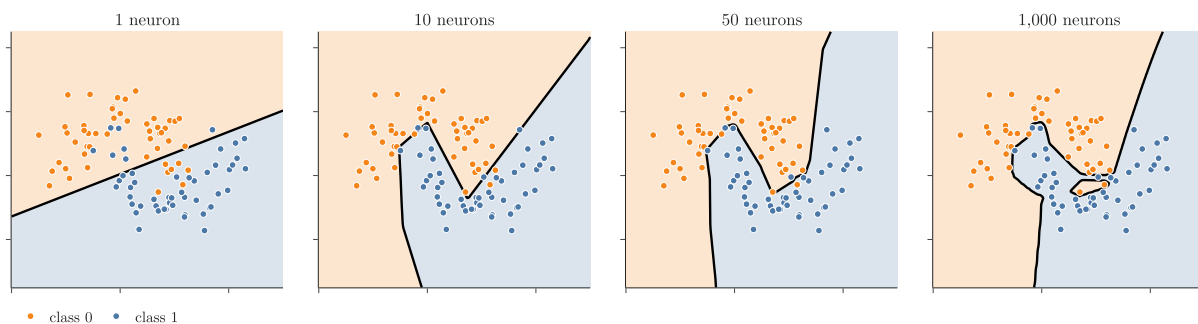


Figure 2: One hidden layer with 1, 10, 50 and 1,000 ReLU neurons, trained for 1,000 epochs on the same data.

Hidden neurons	Training accuracy	Decision boundary
1	86%	a straight line
10	94%	a few straight segments

Hidden neurons	Training accuracy	Decision boundary
50	96%	more segments, following the moons
1,000	100%	bends and pockets around single points

Why does the decision boundary bend more as we add neurons? Figure 3 shows it step by step on the same 100 points.

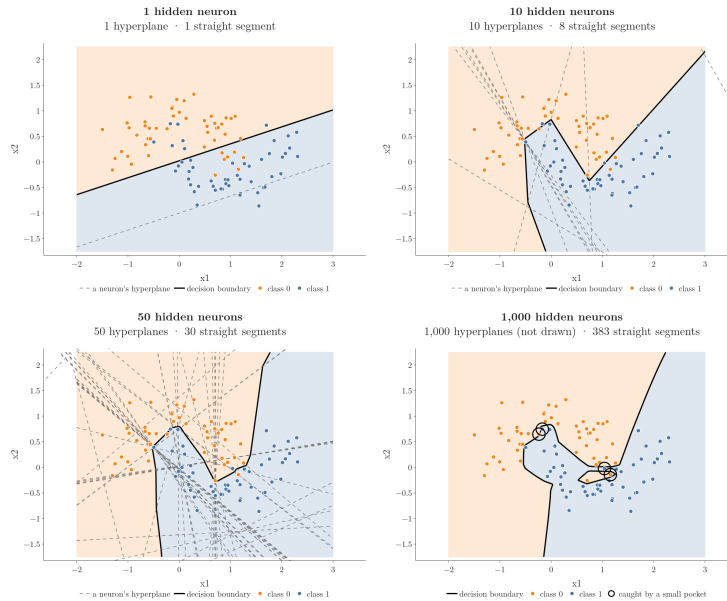


Figure 3: The same four networks. Dashed grey: each hidden neuron’s hyperplane. Black: the decision boundary, which is straight between dashed lines and bends only where it meets one. With 1,000 neurons (lines not drawn) the boundary wraps small pockets around single training points (circled).

- **The line that separates the two classes.** The black line in every panel splits the plane into an orange side and a blue side. The model predicts class 0 on one side and class 1 on the other. This separating line is the **decision boundary** (G-555).
- **What one hidden neuron does.** A neuron with the **ReLU** activation (G-1668) outputs 0 on one side of a straight line and a growing positive value on the other side. That straight line, $\mathbf{w} \cdot \mathbf{x} + b = 0$, is the neuron’s **hyperplane** (G-911): the dashed grey lines in Figure 3.
- **1 neuron.** The output can only turn one neuron’s value up or down, so the decision boundary is a single straight line, parallel to that neuron’s hyperplane (top left). A perceptron gives the same kind of boundary.
- **Many neurons.** Between the dashed lines no neuron switches on or off, so the decision boundary runs straight there. It can change direction only where it meets a dashed line. A boundary made of straight segments joined at bends is **piecewise linear** (G-1496; Goodfellow et al. 2016, §6.3.1 and §6.4.1).

- **Counting the segments.** In our runs the decision boundary has 1, 8, 30 and 383 straight segments for 1, 10, 50 and 1,000 neurons (Notebook). With 10 or 50 neurons, a few dozen segments are enough to follow the two moons.
- **Too many segments.** With 1,000 neurons there are so many places to bend that the decision boundary wraps small pockets around single training points. The four circled points are fitted only by the 1,000-neuron network. The model has learned the noise in these 100 points, not the moon shape: this is **overfitting** (G-1429).

How many different shapes a model can fit is its **capacity** (G-344). More hidden neurons mean more capacity, and a model with more capacity than the data needs “can overfit by memorizing properties of the training set” (Goodfellow et al. 2016, §5.2).

So one way to fix overfitting is to remove nodes. Removing does not have to be literal: if a node’s weights are pushed close to 0, the node barely contributes, and the network behaves like a smaller, simpler one. Pushing weights towards 0 is what regularisation does.

4. Ways to reduce overfitting

Key point: Two routes: more data (collect it, or create it by data augmentation), or a simpler model (dropout, early stopping, regularisation).

Route	Technique	Note
More data	collect more observations	
More data	data augmentation	uniform and log-normal Note
Simpler model	dropout	dropout Note
Simpler model	early stopping	early stopping Note
Simpler model	L1, L2 or L1 + L2 regularisation	this Note

More data helps because the network sees the bigger picture and stops focusing on small details. Data is costly, though, so we often create extra observations (records, rows of the data table) from existing ones: data augmentation (see the [uniform and log-normal Note](#)) makes changed copies of examples, such as cropped, flipped or rotated images of a dog. Data augmentation is used mostly with images and convolutional networks.

There are three kinds of regularisation: L1, L2, and both together. In deep learning, L2 is used almost always and usually gives better results than L1; this Note focuses on L2 and shows L1 for comparison.

5. The penalty term

Key point: Cost = usual cost + $\lambda/(2n) \times$ (sum of all squared weights). Biases are not included.

5.1 L2 regularisation

Key point: Add the sum of every squared weight in the network, times $\lambda/(2n)$, to the cost.

Training finds the weights and biases that minimise a cost function: the average of the loss over the n observations, such as the mean squared error for regression or binary cross-entropy for classification (see the [loss functions Note](#)). L2 regularisation adds a penalty term to it.

1. **In words:** add the sum of the squares of all k weights of the network, multiplied by $\lambda/(2n)$.
2. **Formula:**

$$J = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{y}_i) + \frac{\lambda}{2n} \sum_{j=1}^k w_j^2$$

3. **Example:** a network with 10 weights w_1 to w_{10} adds $\lambda/(2n)(w_1^2 + w_2^2 + \dots + w_{10}^2)$. With $\lambda = 0.03$, $n = 100$ observations and the weights 0.5, -1, 2 and 0.1 (the others 0):

$$\frac{0.03}{200}(0.25 + 1 + 4 + 0.01) = 0.00015 \times 5.26 = 0.00079$$

Some points about this formula:

- λ is a hyperparameter. Larger λ means stronger regularisation; too large, and the network moves from overfitting to underfitting. With $\lambda = 0$ the penalty vanishes and we are back to the plain cost.
- n is the number of observations. The 2 is only for convenience: it cancels when we differentiate. Some books leave it out.
- **Biases are never penalised**, only weights.

In a network the weights live in layers, so the sum is often written per layer. With $w_{ij}^{(l)}$ the weight from node i of layer $l - 1$ to node j of layer l , and L layers:

$$\frac{\lambda}{2n} \sum_{l=1}^L \sum_i \sum_j (w_{ij}^{(l)})^2$$

The per-layer form is the same sum, every weight squared once; it just matches how the weights are stored, which makes it the natural form in code.

5.2 L1 regularisation and both together

Key point: L1 uses absolute values instead of squares and gives a sparse model; L1 + L2 combines both.

L1 regularisation replaces the squares with absolute values, the L1 norm of the weights:

$$J = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{y}_i) + \frac{\lambda}{2n} \sum_{j=1}^k |w_j|$$

As with Lasso, L1 can push weights to exactly 0, giving a sparse model (see the [Lasso sparsity Note](#)); in a network, a node whose weights are all 0 is eliminated. L2, like Ridge, makes weights small but never exactly 0 (see the [ridge key points Note](#), section 2). Using both penalties together is the idea of Elastic Net (see the [Elastic Net Note](#)).

6. Why the weights shrink: weight decay

Key point: With the L2 penalty, every update first multiplies each weight by $1 - \eta\lambda$, a number just below 1, then takes the usual step. The weights decay towards 0.

The penalty is added to the loss, but how does that make the weights small? Look at the update of one weight w . For one observation, the new loss is $L' = L + (\lambda/2) \sum_j w_j^2$ (no n , since this is the loss of a single observation, not the cost). Differentiating the penalty with respect to w leaves only its own term: $(\lambda/2) \cdot 2w = \lambda w$. Rearranged, the update becomes the same as for Ridge (see the [ridge gradient descent Note](#), section 2.3):

1. **In words:** first shrink the weight by the factor $1 - \eta\lambda$, then take the ordinary gradient descent step.

2. **Formula:**

$$w_{\text{new}} = w_{\text{old}} - \eta \left(\frac{\partial L}{\partial w} + \lambda w_{\text{old}} \right) = (1 - \eta\lambda) w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

3. **Example:** with learning rate $\eta = 0.1$ and $\lambda = 0.03$, the factor is $1 - 0.1 \times 0.03 = 0.997$. A weight of 2 becomes $0.997 \times 2 = 1.994$ before the usual step is subtracted.

The second form is the update without regularisation, except that w_{old} is first multiplied by $1 - \eta\lambda$. Since η and λ are positive, this factor is below 1, and it acts at every update of every epoch. So the weights keep moving towards 0. They get very small but never reach exactly 0.

Because the weight shrinks by a fixed factor at each step, L2 regularisation is often called weight decay in neural networks, and $1 - \eta\lambda$ the weight decay factor.

Extra: Strictly, weight decay means multiplying the weights by $1 - \eta\lambda$ directly, outside the gradient. With plain gradient descent this is exactly L2 regularisation, as shown above. With Adam it is not: Adam divides each weight's step by the size of that weight's recent gradients, and with L2 the penalty's gradient gets divided too, so the penalty acts more strongly on some weights than on others (Loshchilov and Hutter 2019, §2). They showed that true weight decay works better with Adam; Keras offers it as `keras.optimizers.AdamW`, and every Keras optimizer accepts a `weight_decay` argument (Keras docs, Optimizers).

7. Regularisation in Keras

Key point: Pass `kernel_regularizer=regularizers.L2(0.03)` to each hidden Dense layer. The boundary becomes smooth, the validation loss stops rising, and the weights shrink into the range -0.5 to 0.5 .

7.1 The data and the network

Key point: 100 points of `make_moons` with noise 0.25; two hidden layers of 128 ReLU nodes; 2,000 epochs.

The data is `make_moons` from scikit-learn: 100 points in two noisy half-moons. Each point is an **observation** (one record of the data); its two coordinates are the **features** (input variables), and its moon, 0 or 1, is the **target** (the output we predict). The network has an input layer of 2 nodes, two hidden layers of 128 ReLU nodes and one sigmoid output node:

Layer	Parameters
Hidden layer 1	$2 \times 128 + 128 = 384$
Hidden layer 2	$128 \times 128 + 128 = 16,512$
Output layer	$128 + 1 = 129$
Total	17,025

Seventeen thousand parameters for 100 points: plenty of room to overfit. The network is compiled with Adam (learning rate 0.01) and binary cross-entropy, and trained for 2,000 epochs, deliberately long, with `validation_split=0.2`.

7.2 Without regularisation

Key point: 100% training accuracy, a boundary full of pockets, and a validation loss that climbs to 1.28.

The network reaches 100% training accuracy. Its decision boundary (Figure 1, left) twists into pockets and narrow fingers to capture single points: clear overfitting. The training curves show it too (Figure 4, left): the training loss drops to almost 0 while the validation loss is lowest at epoch 14 and then rises steadily, to 1.28 after 2,000 epochs.

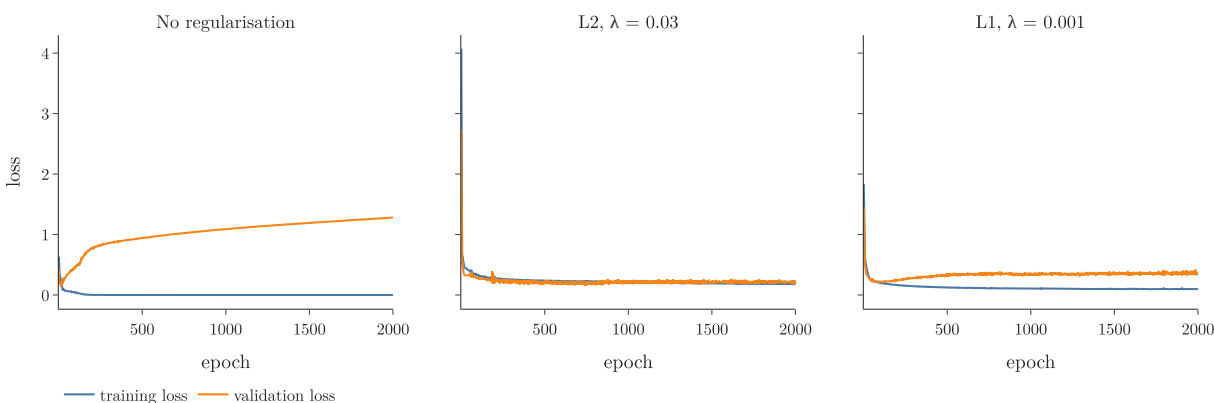


Figure 4: Training and validation loss over 2,000 epochs: without regularisation, with L2 and with L1.

7.3 With L2

Key point: Only `kernel_regularizer` is new; training and validation loss now run side by side.

Python: The same network with L2 regularisation.

```
from keras import regularizers
```

```
model = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(128, activation="relu",
                        kernel_regularizer=regularizers.L2(0.03)),
    keras.layers.Dense(128, activation="relu",
                        kernel_regularizer=regularizers.L2(0.03)),
    keras.layers.Dense(1, activation="sigmoid"),
])
```

The **kernel** is Keras' name for a layer's weight matrix; `kernel_regularizer` penalises the weights only, which matches the rule that biases are not penalised. A separate `bias_regularizer` exists but is rarely used.

The decision boundary (Figure 1, middle) is now a clean shape of a few straight segments that follows the two moons, which should do much better on new data. In Figure 4 (middle) the training and validation curves move side by side for all 2,000 epochs:

	Training accuracy	Validation accuracy	Final validation loss
No regularisation	100%	95%	1.28
L2, $\lambda = 0.03$	95%	95%	0.21
L1, $\lambda = 0.001$	96%	95%	0.34

The validation set has only 20 points, so the accuracies are equal (19 of 20 right). The validation loss shows the difference: without regularisation the network is confidently wrong on its mistakes; with L2 it is not.

Extra: Keras' `L2(0.03)` adds $0.03 \times \sum w^2$ to the loss of each batch: no $1/2$ and no $1/n$. The Notebook checks this on a small matrix: for the weights 0.5, -1 , 2, 0.1 the penalty is $0.03 \times 5.26 = 0.158$. So a Keras λ is not the same number as the λ of the formula in section 5; when moving a value between books and code, check which convention is used.

7.4 The weights shrink

Key point: Without regularisation the 256 first-layer weights range from -2.45 to 2.85 ; with L2, from -0.52 to 0.46 , most of them close to 0.

`model.get_weights()` returns every weight and bias array of the network. The first one, `get_weights()[0]`, holds the weights of the first hidden layer, with shape $(2, 128)$: 2 inputs

times 128 nodes, 256 weights.

Python: Reading the first-layer weights.

```
W1 = model.get_weights()[0]    # shape (2, 128)
w = W1.reshape(256)           # one flat array of 256 weights
w.min(), w.max()
```

Figure 5 compares the two networks.

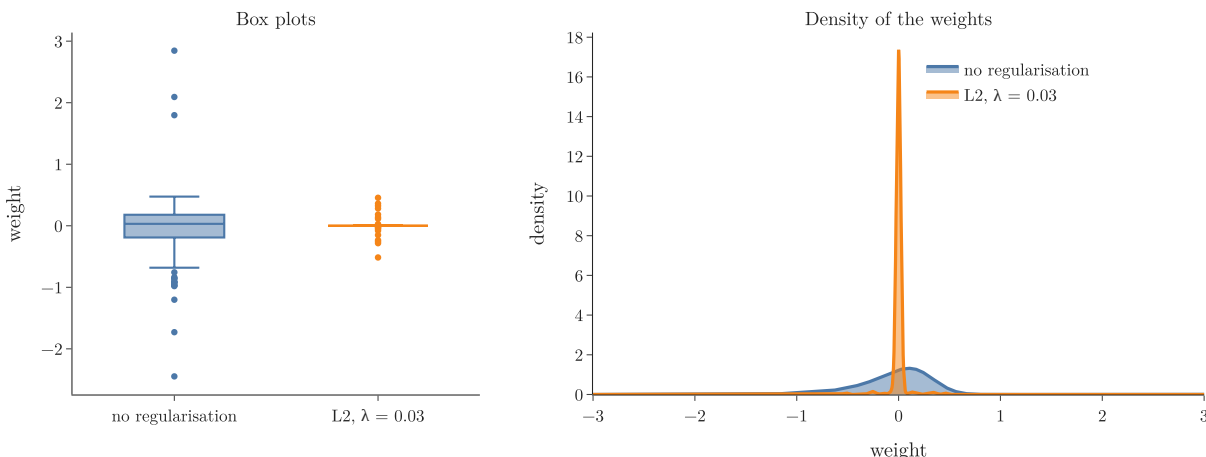


Figure 5: The 256 first-layer weights without regularisation and with L2. Left: box plots. Right: density curves.

	Smallest weight	Largest weight
No regularisation	-2.45	2.85
L2, $\lambda = 0.03$	-0.52	0.46

Without regularisation, 90% of the weights lie between about -0.8 and 0.4 , with outliers out to -2.45 and 2.85 . With L2 the whole range has shrunk: the box collapses to a line at 0, and the density curve (orange) is one tall peak at 0, while the unregularised one (blue) is spread out. The weights have decayed.

Figure 6 shows the decay as it happens. Both networks start from the same weights, all between -0.21 and 0.21 . Watch the blue weights spread out as the network fits the training points, while the orange ones are pulled back into a tall spike at 0. With L2 the largest weight grows to 0.68 by epoch 300, then shrinks to 0.52 by epoch 2,000: the penalty keeps pulling it back.

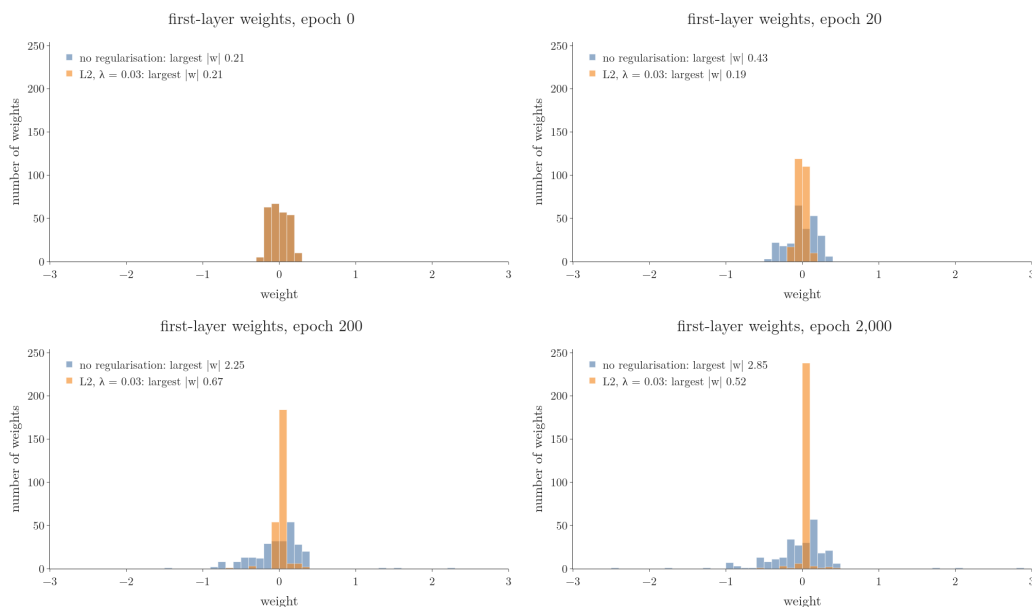


Figure 6: The 256 first-layer weights during training, same start: without regularisation (blue) they spread out, up to 2.85 in size; with L2, $\lambda = 0.03$ (orange), they stay in a spike around 0

Section 7.6 asks whether any of these weights become exactly 0.

7.5 L1

Key point: Swap L2 for L1; λ needs retuning. Here $\lambda = 0.001$ works, with a little more overfitting than L2.

Switching to L1 only needs `regularizers.L1(...)` in place of `regularizers.L2(...)`. The L1 penalty needs its own value of λ , found by trying a few (hyperparameter tuning). With $\lambda = 0.001$ (Figure 1, right) the boundary is clean, but the validation loss creeps up from about epoch 100 (Figure 4, right): a little more overfitting than with L2. Its first-layer weights range from -1.88 to 1.19 .

7.6 Sparse or only small: L1 against L2

Key point: With plain gradient descent, L1 sets more than half of the first-layer weights to 0 and keeps the others fairly large. L2 shrinks every weight but leaves almost none at 0.

Section 5.2 said that L1 makes weights exactly 0 and L2 only makes them small. The reason is the size of the pull towards 0:

- **L2** pulls each weight in proportion to its size (section 6). As a weight gets small, so does the pull, so the weight never quite reaches 0.

- **L1** pulls every weight with the same strength, λ , however small the weight already is. So a weight the data does not need is pushed all the way to 0.

Think of walking towards a wall. L2 halves the remaining distance at every step: we get very close, but never touch the wall. L1 takes a step of fixed length every time: we arrive. The constant pull is the textbook reason why the Lasso gives sparse models and Ridge does not (ESL §3.4.3; Goodfellow et al. 2016, §7.1.2).

To see this cleanly, we train the same network on the same data with plain gradient descent (`keras.optimizers.SGD`, learning rate 0.01) instead of Adam, once with L2(0.03) and once with L1(0.003). Only the penalty changes. The numbers are averages over 3 random starts.

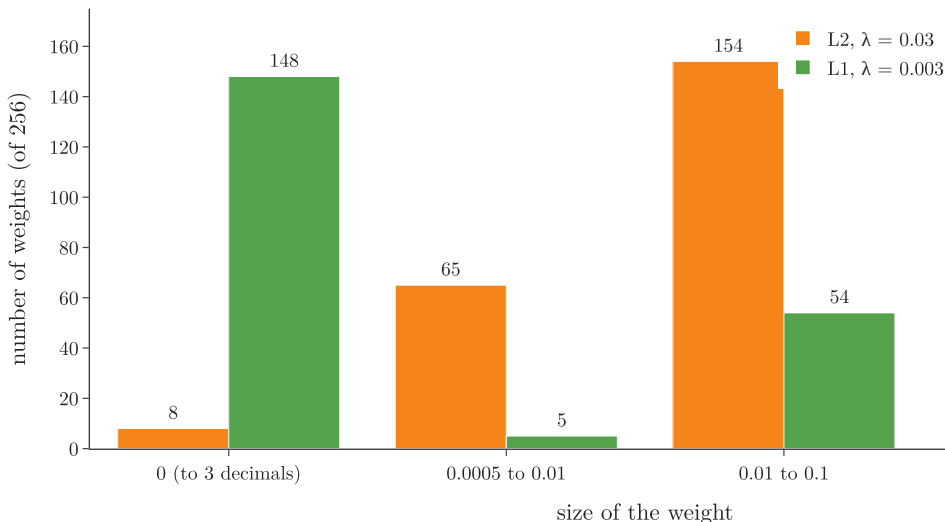


Figure 7: The 256 first-layer weights after 2,000 epochs of plain gradient descent, grouped by size: L2 (orange) and L1 (green), first random start.

	Weights that are 0 (to three decimals)	Largest weight (size)	Validation accuracy
L2, $\lambda = 0.03$	3%	0.24	90%
L1, $\lambda = 0.003$	56%	0.63	90%

Figure 7 shows the difference at a glance. Almost no L2 weight is 0: most are small, between 0.01 and 0.1. The L1 weights split into two groups: more than half are 0, while more of the others stay above 0.1 than with L2 (49 against 29). Both networks score the same 90% on the validation points, but the L1 network does it with fewer than half of its first-layer weights: a sparse model.

Extra: Two details.

1. **“Exactly” 0.** With gradient descent, an L1 weight at 0 still moves by one tiny step, $\eta\lambda = 0.01 \times 0.003 = 0.00003$, back and forth across 0, so it stays within that tiny distance of 0 instead of sitting exactly on it. Lasso solvers reach exact zeros because they set a weight to 0 whenever an update would cross 0, called soft thresholding (Friedman et al. 2010). In a network, the same effect comes from pruning: setting the tiny weights to 0 after training (Han et al. 2015).
2. **Why not Adam?** With Adam the picture changes, and it can even reverse. In section 7.4, with Adam, 92% of the L2 network’s first-layer weights ended below 0.001, against 49% for L1. Adam moves every weight by about the learning rate at every step, whatever the size of the gradient (Kingma and Ba 2015, §2.1). So even L2’s small pull on an unneeded weight gives a full-size step towards 0, while an L1 weight overshoots 0 by a full step and keeps swinging around it. Loshchilov and Hutter (2019, §2) describe the cause: under Adam, the penalty’s gradient is divided by each weight’s past gradient size, so weights with large gradients are regularised less and weights with small gradients relatively more. The Notebook confirms that only the optimizer is responsible:

Optimizer	L2: weights below 0.001	L1: weights below 0.001
SGD, $\eta = 0.01$	7%	19%
Adam, $\eta = 0.01$	92%	49%

(This check uses `L1(0.001)`, the value of section 7.5, so its SGD numbers differ from the table above.) Adam’s uneven treatment of L2 is one reason Keras offers AdamW, with true weight decay (section 6).

8. Summary

	Penalty added to the cost	Effect on weights	In deep learning
L2	$\lambda/(2n) \sum w^2$	shrink towards 0, never exactly 0 (weight decay)	the usual choice
L1	$\lambda/(2n) \sum w $	many pushed to 0: a sparse model	less used
L1 + L2	both	both	occasionally

- Networks overfit because many neurons can draw many small pieces of boundary.
- More data, dropout, early stopping and regularisation all reduce overfitting.
- L2 regularisation multiplies every weight by $1 - \eta\lambda$ before each update; biases are not penalised.
- In Keras: `kernel_regularizer=regularizers.L2( $\lambda$ )` on each hidden layer. Here it turned a validation loss of 1.28 into 0.21 and shrank the weights from ± 2.8 to ± 0.5 .

9. Sources

Built from

- CampusX, “Regularization in Deep Learning | L2 Regularization in ANN | L1 Regularization | Weight Decay in ANN”, YouTube, <https://www.youtube.com/watch?v=4xRonrhtkzc>

Other references

- Loshchilov and Hutter, “Decoupled Weight Decay Regularization”, ICLR 2019, §2 (L2 regularisation and weight decay differ under Adam).
- Kingma and Ba, “Adam: A Method for Stochastic Optimization”, ICLR 2015, Algorithm 1 and §2.1 (the step size is about the learning rate).
- Keras documentation, Optimizers, keras.io/api/optimizers (`weight_decay` argument of every optimizer; **AdamW**).
- Friedman, Hastie and Tibshirani, “Regularization Paths for Generalized Linear Models via Coordinate Descent”, *Journal of Statistical Software*, 2010 (soft thresholding).
- Han, Pool, Tran and Dally, “Learning both Weights and Connections for Efficient Neural Networks”, NeurIPS 2015 (pruning small weights).
- Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, 2nd ed., 2009, §3.4.3 (why the Lasso gives zeros and Ridge does not).
- Goodfellow, Bengio and Courville, *Deep Learning*, MIT Press, 2016, §5.2 (capacity, overfitting and underfitting), §6.3.1 and §6.4.1 (rectified linear units give piecewise linear functions), §7.1.1 (L2, weight decay) and §7.1.2 (L1 and sparsity).

10. Key terms

Term	Meaning
Decision boundary	The line or curve separating the inputs a classifier assigns to different classes
Hyperplane	A flat boundary $\mathbf{w} \cdot \mathbf{x} + b = 0$; a straight line when there are two features
Piecewise linear	Made of straight segments joined at bends; the shape of a ReLU network's decision boundary
Capacity	A model's ability to fit a wide variety of functions; too much capacity for the data leads to overfitting
Penalty term	The extra term added to the cost to discourage large weights
Weight decay factor	$1 - \eta\lambda$, the factor by which L2 regularisation shrinks every weight at each update
<code>kernel_regularizer</code>	Keras <code>Dense</code> setting that adds an L1 or L2 penalty on the layer's weights
Kernel	Keras' name for a layer's weight matrix
<code>get_weights</code>	Keras method that returns every weight and bias array of a model
AdamW	Adam with true weight decay applied directly to the weights, not through the loss